

System Security Report

Gebeya E-Commerce Platform Security Overview

*Multi-Factor Authentication, CSRF Protection, Input Sanitisation,
Password Security, Rate Limiting, and Role-Based Access Control.*

Overview

This report documents the security controls implemented in the Gebeya e-commerce platform. Gebeya is a PHP/MySQL application running on XAMPP that serves three user roles buyer, seller, and admin each requiring full Multi-Factor Authentication before accessing any protected resource. The security design follows a defence-in-depth approach: each protection layer is independent so a failure in one does not automatically compromise the others.

The report covers authentication and session security, password handling, CSRF protection, input validation and XSS prevention, SQL injection prevention, network and header security, data protection, rate limiting, role-based access control, and file upload security. Code excerpts from the live codebase are included throughout, and figure placeholders indicate where screenshots of the relevant code sections should be inserted.

Authentication and Session Management

Multi-Factor Authentication Flow

Authentication proceeds in two distinct phases. The first factor is a correct email address and password. On success the server does not establish a full session; instead it writes a `pending_mfa_user_id` staging key to the session. A 6-digit one-time password is generated, hashed, stored with a 5-minute expiry, and dispatched by email. The second factor is entering that OTP within the expiry window. Only after successful OTP verification is `$_SESSION['user_id']` set, establishing a fully authenticated session.

Session Staging login.php

```
// MFA OTP GENERATION
$otp = (string)random_int(100000, 999999);
$otp_hash = password_hash($otp, PASSWORD_DEFAULT);
$expires = date('Y-m-d H:i:s', time() + 300); // 5 minutes expiry

$upd = mysqli_prepare($conn, "UPDATE users SET otp_hash = ?, otp_expires_at = ? WHERE user_id = ?");
mysqli_stmt_bind_param($upd, "ssi", $otp_hash, $expires, $user_id);
mysqli_stmt_execute($upd);
mysqli_stmt_close($upd);

// Send Email
send_otp_email($email, $otp);

// Set pending MFA session
$_SESSION["pending_mfa_user_id"] = (int)$user_id;
session_regenerate_id(true);

// Reset login attempts on successful password verification
$_SESSION['login_attempts'] = 0;
unset($_SESSION['last_login_attempt']);

redirect('/public/verify_otp.php');
```

Figure 1: MFA session staging after password verification a cryptographically generated OTP is bcrypt-hashed and stored with a 5-minute expiry, dispatched by email, and the user is placed in a `pending_mfa_user_id` session key with the session ID regenerated before redirect.

OTP Security Properties

- Generated with `random_int(100000, 999999)` PHP's cryptographically secure integer generator backed by the OS CSPRNG.
- Immediately hashed with `password_hash(PASSWORD_DEFAULT)` before storage; the plain-text OTP never persists in the database.
- Expires after 5 minutes; `otp_expires_at` is checked on the server with `strtotime()` before `password_verify()` is called.
- `otp_hash` and `otp_expires_at` are set to NULL in the database immediately after successful verification, preventing replay attacks.
- A resend throttle (`$_SESSION['last_otp_sent']`) prevents a new OTP being dispatched within 60 seconds of the previous one.

```
// Rate limit OTP attempts: max 3 per 5 minutes
if (!isset($_SESSION['otp_attempts'])) {
    $_SESSION['otp_attempts'] = 0;
    $_SESSION['first_otp_attempt'] = time();
}

if (time() - $_SESSION['first_otp_attempt'] > 300) {
    // Reset after 5 mins
    $_SESSION['otp_attempts'] = 0;
    $_SESSION['first_otp_attempt'] = time();
}
```

Figure 2: OTP attempt rate-limiting block initialising the attempt counter and 5-minute window, resetting the counter after the window expires, and blocking further submissions after 3 failed attempts.

Session Security

PHP sessions are hardened through settings applied in `includes/config.php` before any output is sent:

```
<?php
// includes/config.php

// Secure Session Cookie Settings
ini_set('session.cookie_httponly', 1);
ini_set('session.cookie_samesite', 'Strict');
// ini_set('session.cookie_secure', 1); // Uncomment if using HTTPS

// Global Security Headers
header("X-Content-Type-Options: nosniff");
header("X-Frame-Options: DENY");
header("X-XSS-Protection: 1; mode=block");
header("Strict-Transport-Security: max-age=31536000; includeSubDomains");

// Use RELATIVE base URL so it works on any localhost port
define('BASE_URL', '/gebeya');
```

Figure 3: Centralised security configuration setting `HttpOnly` and `SameSite=Strict` session cookie flags and emitting all four HTTP security headers on every response before any page output.

- Session ID is regenerated with `session_regenerate_id(true)` at three points: after password verification (before OTP step), after successful OTP verification (when granting access), and on logout.
- On logout, `$_SESSION` is set to an empty array, the session cookie is explicitly expired by reissuing it with a negative max-age timestamp, and `session_destroy()` is called. A new session with a fresh ID is then started immediately.
- The `session.cookie_samesite = Strict` setting prevents the session cookie from being attached to any cross-origin request, providing a browser-level CSRF defence in addition to the token-based protection.

Password Security

Complexity Requirements

Passwords are required to be at least 8 characters long and must satisfy four additional checks enforced by server-side regex validation on every registration request:

- At least one uppercase letter: `preg_match('/[A-Z]/', $password)`
- At least one lowercase letter: `preg_match('/[a-z]/', $password)`
- At least one digit: `preg_match('/[0-9]/', $password)`
- At least one special character: `preg_match('/[^a-zA-Z0-9]/', $password)`

```
// Password complexity check
if (strlen($password) < 8) {
    $errors[] = "Password must be at least 8 characters.";
} elseif (!preg_match('/[A-Z]/', $password) || !preg_match('/[a-z]/', $password) || !preg_match('/[0-9]/', $password) || !preg_match('/[^a-zA-Z0-9]/', $password)) {
    $errors[] = "Password must contain at least one uppercase letter, one lowercase letter, one number, and one special character.";
}

if (!in_array($account_type, ["buyer", "seller"], true)) $errors[] = "Invalid account type.";
```

Figure 4: Password complexity validation enforcing a minimum length of 8 characters and four character-class requirements via `preg_match`, with a strict `in_array` whitelist check on the submitted account type.

Hashing

Passwords are hashed with `password_hash()` using `PASSWORD_DEFAULT` (bcrypt with the default cost factor). The resulting hash is stored in the `password_hash` column; the plaintext password is never persisted. `password_verify()` is used for comparison during login, which is timing-safe and prevents timing-based enumeration attacks.

The same hashing functions are used for the OTP; `password_hash` for storage and `password_verify` for comparison meaning no secret value is ever stored in plain text in the database.

CSRF Protection

CSRF protection is implemented through a synchroniser token pattern managed by three functions in `includes/auth.php`:

- `generate_csrf_token()`; checks `$_SESSION['csrf_token']`; if absent, generates a new token with `bin2hex(random_bytes(32))` (64 hex characters, 256 bits of entropy) and stores it in the session.
- `verify_csrf_token($token)`; compares the submitted token against the session value using `hash_equals()`, which is timing-safe and prevents length-based timing attacks.
- `check_csrf()`; reads `$_POST['csrf_token']`, calls `verify_csrf_token()`, and redirects with a flash error if verification fails.

```

function generate_csrf_token(): string {
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

1 reference
function verify_csrf_token(string $token): bool {
    if (empty($_SESSION['csrf_token']) || !hash_equals($_SESSION['csrf_token'], $token)) {
        return false;
    }
    return true;
}

14 references
function check_csrf(): void {
    $token = $_POST['csrf_token'] ?? '';
    if (!verify_csrf_token($token)) {
        flash_set('danger', 'Invalid security token. Please refresh the page and try again.');
```

Figure 5: CSRF token functions generating a 256-bit token with `bin2hex(random_bytes(32))`, verifying it with the timing-safe `hash_equals`, and redirecting with a flash error on any mismatch.

Every HTML form that performs a state-changing action includes a hidden CSRF field:

Example from `public/login.php`; hidden CSRF field in every POST form:

```
<input type="hidden" name="csrf_token" value="<?= e(generate_csrf_token()) ?>">
```

`check_csrf()` is called at the top of every POST handler; login, register, OTP verify, profile update, cart actions, checkout, category management, listing moderation, and all seller operations. Combined with the `SameSite=Strict` session cookie, the application has two independent CSRF defences.

Cross-Site Scripting (XSS) Prevention

XSS is mitigated through consistent output encoding on every page in the application. The global helper `e()` is defined in `includes/functions.php` and applied to all user-supplied data before it is rendered in HTML:

```
function e(string $s): string {  
    return htmlspecialchars($s, ENT_QUOTES, 'UTF-8');  
}
```

Figure 6: The `e()` helper function applying `htmlspecialchars` with `ENT_QUOTES` and `UTF-8`, shown applied to user-supplied product titles, seller names, and image attributes in the store template.

`ENT_QUOTES` encodes both single and double quote characters in addition to `<` and `>`, making the function safe for use inside HTML attributes as well as element content. The `UTF-8` charset argument prevents double-encoding and multi-byte character exploits.

- Full names submitted on registration are passed through `strip_tags()` and a regex that removes `<script>` and `<style>` tag pairs before storage, reducing XSS risk at the input layer.
- The `X-XSS-Protection: 1; mode=block` header instructs older browsers to enable their built-in XSS filter.

SQL Injection Prevention

All database interaction uses MySQLi prepared statements with parameterised queries. No string concatenation of user input into SQL is present anywhere in the codebase. The `db.php` bootstrap enables `MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT` so any query error throws a `mysqli_sql_exception` rather than silently failing or returning misleading results.

```
$stmt = mysqli_prepare($conn, "SELECT user_id, password_hash, status FROM users WHERE email = ? LIMIT 1");  
mysqli_stmt_bind_param($stmt, "s", $email);  
mysqli_stmt_execute($stmt);
```

Figure 7: Parameterised user lookup using a `?` placeholder in the SQL string with `mysqli_stmt_bind_param` type `'s'`, ensuring the email value is never concatenated into the query.

Dynamic `IN()` clauses used in the cart to look up multiple product IDs at once are constructed by building a string of `?` placeholders equal to the count of IDs, then binding all values in a single call using the spread operator:

```

if ($cart) {
    $ids = array_keys($cart);
    $placeholders = implode(',', array_fill(0, count($ids), '?'));
    $types = str_repeat('i', count($ids));

    $sql = "SELECT product_id, title, price, stock, status
            FROM products
            WHERE product_id IN ($placeholders)";

    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, $types, ...$ids);
    mysqli_stmt_execute($stmt);
    $res = mysqli_stmt_get_result($stmt);
}

```

Figure 8: Safe dynamic IN() clause construction using `array_fill` and `implode` to build placeholders, `str_repeat` to build the types string, and `bind_param` with the spread operator to bind all product IDs without any interpolation.

Security Headers

Security headers are emitted by `includes/config.php`, which is the first file included on every page. Because it runs before any output, the headers are applied universally including to error responses and redirects.

Header	Purpose
X-Content-Type-Options: nosniff	Prevents the browser from MIME-sniffing the response type, blocking MIME confusion attacks
X-Frame-Options: DENY	Prevents the application from being embedded in any iframe, blocking clickjacking on all pages
X-XSS-Protection: 1; mode=block	Instructs legacy browsers to enable their built-in XSS filter and block the page on detection
Strict-Transport-Security: max-age=31536000; includeSubDomains	Instructs browsers to use HTTPS for one year including subdomains (effective once HTTPS is configured)

The `.htaccess` file at the project root blocks direct HTTP access to the `.env` file containing database credentials and SMTP secrets:


```
<?php
// includes/config.php

// Secure Session Cookie Settings
ini_set('session.cookie_httponly', 1);
ini_set('session.cookie_samesite', 'Strict');
// ini_set('session.cookie_secure', 1); // Uncomment if using HTTPS

// Global Security Headers
header("X-Content-Type-Options: nosniff");
header("X-Frame-Options: DENY");
header("X-XSS-Protection: 1; mode=block");
header("Strict-Transport-Security: max-age=31536000; includeSubDomains");

// Use RELATIVE base URL so it works on any localhost port
define('BASE_URL', '/gebeya');
```

Figure 9: All four HTTP security headers emitted from a single centralised location, covering MIME sniffing prevention, clickjacking protection, XSS filter activation, and HSTS enforcement.

Rate Limiting and Attempt Controls

Login Rate Limiting

Login attempts are tracked per session using `$_SESSION['login_attempts']` and `$_SESSION['last_login_attempt']`. After 5 consecutive failed attempts within a 5-minute window the handler rejects further submissions. The counter resets on a successful password verification or after the 5-minute window expires.

```
// Basic Rate Limiting
$max_attempts = 5;
$lockout_time = 300; // 5 minutes

if (isset($_SESSION['login_attempts']) && $_SESSION['login_attempts'] >= $max_attempts) {
    if (time() - $_SESSION['last_login_attempt'] < $lockout_time) {
        $errors[] = "Too many failed attempts. Please wait 5 minutes before trying again.";
    } else {
        $_SESSION['login_attempts'] = 0; // reset after lockout
    }
}
```

Figure 10: Session-scoped login rate limiter declaring a 5-attempt limit over a 5-minute window, blocking further attempts within the lockout period, and incrementing the counter and timestamp on every failed submission.

OTP Attempt Limiting

OTP verification is capped at 3 attempts per 5-minute window using `$_SESSION['otp_attempts']` and `$_SESSION['first_otp_attempt']`. This limit is always enforced with no bypass. After 3 failed attempts the form locks and the user must request a new code, which itself is throttled to one resend per 60 seconds via `$_SESSION['last_otp_sent']`.

```
// Check if we can resend (very basic rate limit)
if (isset($_SESSION['last_otp_sent']) && (time() - $_SESSION['last_otp_sent']) < 60) {
    $errors[] = "Please wait at least 60 seconds before requesting a new code.";
}
```

Figure 11: OTP resend throttle preventing a new code within 60 seconds, and attempt counter capping verification at 3 tries per 5-minute window with automatic counter reset after the window expires.

Role-Based Access Control

Gebeya implements role-based access control through guard functions defined in `includes/auth.php`. Every protected page calls one of these functions before any business logic or output:

- `require_login()`: redirects to login if no valid `user_id` is present in the session.
- `require_role($conn, $role_name)`: calls `require_login()` first, then queries `user_roles` joined to `roles` to verify the user holds the specified role. If not, the user is redirected to their own dashboard.
- `require_buyer_only($conn)`: explicitly blocks admin and seller accounts from buyer pages such as cart and checkout.

The `has_role()` function performs a parameterised SQL query on every request an attacker cannot escalate privileges by manipulating session data, as the role check always hits the database:

```
function has_role(mysqli $conn, string $role): bool {
    if (session_status() === PHP_SESSION_NONE) {
        session_start();
    }

    if (empty($_SESSION['user_id'])) {
        return false;
    }

    $uid = (int)$_SESSION['user_id'];

    $sql = "SELECT 1
            FROM user_roles ur
            JOIN roles r ON r.role_id = ur.role_id
            WHERE ur.user_id = ? AND r.role_name = ?
            LIMIT 1";

    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, "is", $uid, $role);
    mysqli_stmt_execute($stmt);
    mysqli_stmt_store_result($stmt);

    $ok = (mysqli_stmt_num_rows($stmt) > 0);
    mysqli_stmt_close($stmt);

    return $ok;
}
```

Figure 12: `has_role()` performing a live parameterised `SELECT 1 JOIN` query on `user_roles` and `roles` on every protected request, preventing privilege escalation through session manipulation.

Ownership is enforced at the query level throughout the application. Seller pages include `AND seller_id = ?` in every `SELECT`, `UPDATE`, and `DELETE` that touches product or order-item data. The address delete handler requires `AND user_id = ?` to prevent cross-account deletion. A fully authenticated user of the wrong account cannot access data they do not own the database query itself enforces the boundary.

```
$stmt = mysqli_prepare(
    $conn,
    "SELECT product_id, seller_id, title, description, price, stock, product_condition, status, image_path, image_alt
    FROM products
    WHERE product_id = ? AND seller_id = ?
    LIMIT 1"
);
mysqli_stmt_bind_param($stmt, "ii", $product_id, $seller_id);
mysqli_stmt_execute($stmt);
$res = mysqli_stmt_get_result($stmt);
$product = mysqli_fetch_assoc($res);
mysqli_stmt_close($stmt);
```

Figure 13: Ownership-enforced product `SELECT` using `WHERE product_id = ? AND seller_id = ?` with both values bound, ensuring a seller can only load and edit their own listings regardless of URL manipulation.

File Upload Security

Product image uploads are accepted only on the seller listing creation and edit pages, both of which are guarded by `require_role($conn, 'seller')`. The upload pipeline applies the following controls in order:

- PHP upload error check: `$_FILES['image']['error']` must equal `UPLOAD_ERR_OK` before any further processing.
- Size limit: files larger than 2MB ($2 * 1024 * 1024$ bytes) are rejected before the file is moved from the temporary directory.
- MIME type detection: `finfo_open(FILEINFO_MIME_TYPE)` reads the binary file header from the temporary path. Only `image/jpeg`, `image/png`, and `image/webp` are accepted. The extension is derived from the server-determined MIME type, not from the client-supplied filename.
- Safe filename: the saved file is named `{product_id}_{timestamp}.{ext}`, preventing directory traversal, name collisions, and PHP execution via uploaded files.
- Atomicity: the file is only moved to `uploads/products/` after the database transaction commits. If the transaction rolls back, the uploaded file is deleted immediately.

```
21 if ($_SERVER["REQUEST_METHOD"] === "POST") {
36     if (!$errors) {
46         try {
77             /* 3) Upload image (optional) AFTER product exists */
78             if (!empty($_FILES['image']['name'])) {
79                 if ($_FILES['image']['error'] !== UPLOAD_ERR_OK) {
80                     throw new Exception("Image upload failed.");
81                 }
82
83                 $tmp = $_FILES['image']['tmp_name'];
84                 $size = (int)$_FILES['image']['size'];
85
86                 if ($size > 2 * 1024 * 1024) {
87                     throw new Exception("Image too large (max 2MB).");
88                 }
89
90                 $finfo = finfo_open(FILEINFO_MIME_TYPE);
91                 $mime = finfo_file($finfo, $tmp);
92                 finfo_close($finfo);
93
94                 $allowed = [
95                     'image/jpeg' => 'jpg',
96                     'image/png'  => 'png',
97                     'image/webp' => 'webp',
98                 ];
99
100                 if (!isset($allowed[$mime])) {
101                     throw new Exception("Invalid image type. Use JPG, PNG, or WebP.");
102                 }
103
104                 $ext = $allowed[$mime];
105
106                 $destDir = __DIR__ . '/../uploads/products';
107                 if (!is_dir($destDir)) {
108                     @mkdir($destDir, 0755, true);
109                 }
110                 if (!is_dir($destDir) || !is_writable($destDir)) {
111                     throw new Exception("Upload folder not writable: " . $destDir);
112                 }
113
114                 // Deterministic-ish name (traceable), avoids random spam duplicates
115                 $name = $product_id . "-" . time() . "." . $ext;
116                 $dest = $destDir . '/' . $name;
117
118                 if (!move_uploaded_file($tmp, $dest)) {
119                     throw new Exception("Could not save uploaded image.");
120                 }
121
122                 $uploadedAbsPath = $dest;
123                 $imagePath = './uploads/products/' . $name;
124                 $imageAlt = $title ?: null;
125
126                 /* 4) Update product with image fields */
127                 $stmt3 = mysqli_prepare($conn,
128                     "UPDATE products SET image_path=?, image_alt=? WHERE product_id=?");
129                 mysqli_stmt_bind_param($stmt3, "ssi", $imagePath, $imageAlt, $product_id);
130                 mysqli_stmt_execute($stmt3);
131                 mysqli_stmt_close($stmt3);
132             }
133         }
134     }
135 }
```

Figure 14: Image upload security pipeline validating the PHP error flag, rejecting files over 2MB, detecting MIME type from the binary file header via `finfo`, generating a safe server-side filename, and saving the file only after a successful database commit.

Data Protection

Secrets Management

All sensitive configuration values database credentials and SMTP credentials are stored in a `.env` file at the project root and parsed by `db.php` at startup. The `.env` file is blocked from direct HTTP access by the `.htaccess` directive. No secret is hardcoded in any PHP file.

```

$env_file = __DIR__ . '/../.env';
if (file_exists($env_file)) {
    $lines = file($env_file, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
    foreach ($lines as $line) {
        if (strpos(trim($line), '#') === 0) continue;
        $parts = explode('=', $line, 2);
        if (count($parts) === 2) {
            $_ENV[trim($parts[0])] = trim($parts[1]);
        }
    }
}

```

Figure 15: Runtime secrets loading by parsing the .env file line by line, skipping comments, splitting on the first =, and writing values to \$_ENV keeping all credentials out of the source code.

Data at Rest

- Passwords: hashed with password_hash(PASSWORD_DEFAULT) before storage; never stored in plain text.
- OTP codes: hashed with password_hash(PASSWORD_DEFAULT) immediately upon generation; the plain-text code exists in memory only for the duration of the email dispatch call.
- No payment card data is stored; the payments table records only the method name and status.

Cart Cookie

The cart is persisted in a browser cookie (gebeya_cart) for convenience across sessions. This cookie contains product IDs and quantities only, no prices or user identifiers. Cart totals and product prices are always recalculated server-side from the database during checkout, so a tampered cookie cannot manipulate order amounts.

Vulnerability Assessment Summary

The following table summarises the security controls implemented across the application.

Control	Implementation
CSRF Protection	Synchroniser token generated with bin2hex(random_bytes(32)), verified with hash_equals(), on every state-changing POST form. Combined with SameSite=Strict session cookie.
SQL Injection	All queries use MySQLi prepared statements with bound parameters. Dynamic IN() clauses use array_fill() placeholder construction. MYSQLI_REPORT_STRICT raises exceptions on query errors.
XSS Prevention	Universal output encoding via htmlspecialchars(ENT_QUOTES, UTF-8) through the e() helper. strip_tags() and regex sanitisation applied to full_name at input. X-XSS-Protection header set.

Password Security	bcrypt (PASSWORD_DEFAULT) for all passwords. Minimum 8 characters with uppercase, lowercase, digit, and special character enforced server-side on registration.
OTP / MFA	Generated with random_int (CSPRNG), bcrypt-hashed before storage, 5-minute expiry, set to NULL after use. 3-attempt cap, 60-second resend throttle.
Session Security	HttpOnly and SameSite=Strict cookie flags set in config.php. session_regenerate_id(true) called after password verify, after OTP verify, and on logout. Full session destruction on logout.
Security Headers	X-Content-Type-Options, X-Frame-Options: DENY, X-XSS-Protection, Strict-Transport-Security emitted on every response via config.php.
.env Protection	.htaccess blocks direct HTTP access to the .env secrets file.
Login Rate Limiting	5 failed attempts trigger a 5-minute session-scoped lockout.
OTP Rate Limiting	3 failed OTP attempts per 5-minute window; resend throttled to once per 60 seconds.
Role-Based Access Control	require_role(), require_login(), require_buyer_only() called at the top of every protected page. Role verified by a live parameterised DB query on every request.
Ownership Enforcement	Seller and buyer data operations include AND seller_id/user_id = ? in every UPDATE and DELETE, enforced at the query level.
File Upload Security	MIME type validated with finfo on binary content (not client filename). Max 2MB. Safe filename generation. File only saved after DB commit; deleted on rollback.
Input Validation	Email: filter_var. Password: 4-rule regex. Account type, product condition, payment method, item status: in_array with strict mode.
Transaction Integrity	Checkout wraps order creation, item insertion, stock decrement, and payment in one MySQLi transaction with automatic rollback on any exception.

Known Limitations

- HTTPS not enforced in development: `session.cookie_secure` is commented out in `config.php`. It must be enabled and an SSL/TLS certificate installed before production deployment.
- Login and OTP rate limiting are session-scoped: an attacker who clears cookies between attempts can reset the counter. A database-backed counter per `user_id` would be more robust.
- Registration has no rate limit: automated account creation is not currently throttled. A rate limiter or CAPTCHA on the registration form is recommended.
- OTP delivery via Mailtrap sandbox: codes are captured in a sandbox that does not forward to real inboxes. A production email provider (SendGrid, AWS SES, or Brevo) must be configured before go-live.
- No Content Security Policy header: a CSP header is not currently set. Adding a strict CSP would provide a browser-level barrier against any script injection that bypasses the output encoder.
- Cart cookie is unsigned: the `gebeya_cart` cookie is JSON-encoded without an HMAC. A user could modify cart contents in the browser, though the checkout handler mitigates this by recalculating all prices server-side.
- No biometric or hardware key support: WebAuthn / FIDO2 is not implemented but could be added as an additional or alternative second factor in a future version.

Conclusion

The Gebeya platform addresses authentication security at multiple independent layers: strong password requirements enforced server-side, bcrypt hashing for all stored secrets, an OTP-based second factor with a short expiry window and attempt limiting, session ID regeneration at each authentication milestone, CSRF tokens on every state-changing form, universal output encoding against XSS, parameterised queries against SQL injection, security headers on every response, and role-based access control with database-level ownership enforcement.

The most significant pre-production action items are enabling HTTPS with the `session.cookie_secure` flag, switching OTP email delivery to a production SMTP provider, moving the login and OTP attempt counters to database-backed storage for session-bypass resistance, adding a rate limit or CAPTCHA to the registration endpoint, and implementing a Content Security Policy header.